



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

300+ L1 Interview Questions For .NET Developers

- C# Fundamentals – 40
- OOP Concepts – 40
- .NET Core Basics – 40
- ASP.NET MVC / Web API – 40
- Entity Framework / LINQ – 40
- SQL / Database – 40
- REST / Microservices – 30
- Common Tools & Best Practices – 30

C# Fundamentals — 50 Questions & Answers

Follow me on LinkedIn: <https://www.linkedin.com/in/sandeepall/>



1. What is the difference between `int` and `Int32` in C#?

Both represent 32-bit signed integers.

- `int` is just an alias for `System.Int32`.
- Functionally they're the same.
Example:

```
int x = 10;  
Int32 y = 20;  
Console.WriteLine(x + y); // 30
```

In short: `int` = shorthand, `Int32` = formal name.

2. What is a namespace in C#?

A **namespace** organizes code into logical groups to avoid naming conflicts.

Example:

```
namespace MyApp.Utilities {  
    class Logger { }  
}
```

You might have another `Logger` in `MyApp.Services` — namespaces keep them separate.

3. What are value types and reference types?

- **Value types** store actual data (int, bool, struct).
- **Reference types** store references to memory locations (class, string, array).
Example:



```
int a = 5;
int b = a; // Copy value
b = 10; // a still 5
```

```
Person p1 = new Person();
Person p2 = p1; // Reference copy
p2.Name = "John"; // p1.Name also changes
```

4. Difference between **const** and **readonly**

- **const** = compile-time constant.
- **readonly** = value set at runtime (in constructor).
Example:

```
const double Pi = 3.14;
readonly DateTime Created = DateTime.Now;
```

5. What is a nullable type in C#?

It allows value types to hold **null**.
Example:

```
int? age = null;
if (age.HasValue)
    Console.WriteLine(age.Value);
```

6. What is boxing and unboxing?

Converting a **value type** to **object** = **Boxing**
Converting it back = **Unboxing**
Example:

```
int num = 10;
```



```
object obj = num; // Boxing
int x = (int)obj; // Unboxing
```

7. Difference between `==` and `.Equals()`

- `==` compares **references** (by default for classes).
- `.Equals()` compares **contents** (can be overridden).
Example:

```
string a = "hello";
string b = new string("hello".ToCharArray());
Console.WriteLine(a == b); // True
Console.WriteLine(a.Equals(b)); // True
```

8. What is the purpose of `var` keyword?

`var` lets the compiler **infer the type** at compile time.
Example:

```
var name = "Alice"; // string
var age = 25; // int
```

9. Difference between `String` and `string`

Both are the same — `string` is an alias for `System.String`.

10. How do you declare an array in C#?

Example:

```
int[] numbers = new int[3] { 1, 2, 3 };
```



11. Implicit vs Explicit conversions

- **Implicit:** automatic (no data loss)
- **Explicit:** requires cast (may lose data)
Example:

```
int x = 10;  
double y = x; // implicit  
int z = (int)y; // explicit
```

12. Difference between Array and ArrayList

- **Array:** fixed size, stores one type.
- **ArrayList:** resizable, stores any type (boxing occurs).
Example:

```
ArrayList list = new ArrayList();  
list.Add(10);  
list.Add("Hello");
```

13. Difference between List<T> and ArrayList

List<T> is **type-safe** and **faster** (no boxing).
Example:

```
List<int> numbers = new List<int> { 1, 2, 3 };
```

14. What are ref and out parameters?



- **ref:** must be initialized before passing.
- **out:** must be assigned inside the method.
Example:

```
void Add(int a, int b, out int sum) { sum = a + b; }
```

15. Difference between for, foreach, and while loops

- **for:** known iterations
- **foreach:** iterates collections
- **while:** until condition false
Example:

```
foreach(var item in list) { ... }
```

16. What is an indexer in C#?

Allows objects to be indexed like arrays.

Example:

```
class MyList {  
    private int[] data = new int[3];  
    public int this[int i] { get => data[i]; set => data[i] = value;  
}  
}
```

17. What is a delegate in C#?

A **delegate** is a type-safe function pointer.

Example:



```
delegate void Greet(string name);  
Greet greet = (n) => Console.WriteLine($"Hello {n}");  
greet("Sam");
```

18. What is an event in C#?

An **event** notifies subscribers when something happens.

Example:

```
event Action OnCompleted;
```

19. Difference between **throw** and **throw ex**

- **throw**: preserves stack trace.
- **throw ex**: resets it.

Always prefer **throw**;

20. Difference between finalize and dispose

- **Finalize**: automatic by GC.
- **Dispose**: manual resource cleanup (IDisposable).

Example:

```
using(var conn = new SqlConnection()) { ... }
```

21. What are generics in C#?

They allow **type-safe reusable code**.

Example:



```
List<int> list = new List<int>();
```

22. What is nullable operator (?.)?

Prevents `NullReferenceException`.

Example:

```
Console.WriteLine(person?.Address?.City);
```

23. What is string interpolation?

Example:

```
string name = "Alex";  
Console.WriteLine($"Hello {name}");
```

24. How to prevent inheritance of a class?

Use `sealed` keyword.

```
sealed class Account { }
```

25. What are extension methods?

Add new methods to existing classes without modifying them.

Example:

```
public static class StringExt {  
    public static bool IsEmpty(this string str) =>  
        string.IsNullOrEmpty(str);  
}
```



26. Difference between dynamic and var

- **var**: compile-time type.
- **dynamic**: runtime type resolution.
Example:

```
dynamic val = 5;  
val = "Hello"; // valid
```

27. What is the using statement in C#?

Automatically disposes objects.
Example:

```
using (var fs = new FileStream("a.txt", FileMode.Open)) { }
```

28. Purpose of static keyword

Belongs to the **type**, not instance.
Example:

```
static class MathUtils { public static int Add(int a, int b) => a +  
b; }
```

29. What are tuples in C#?

Group multiple values.
Example:

```
var person = (Name: "John", Age: 25);
```

30. Difference between IEnumerable and IQueryable



- **IEnumerable:** executes in memory.
- **IQueryable:** executes in database (deferred).
Used often in LINQ to Entities.

31. Difference between is and as operators

- **is:** checks type.
- **as:** safe cast (returns null if fails).
Example:

```
if(obj is Person p) Console.WriteLine(p.Name);
```

32. Difference between struct and class

- **Struct:** value type, stored on stack.
- **Class:** reference type, stored on heap.

33. Shallow copy vs deep copy

- **Shallow:** copies reference.
- **Deep:** copies new object with data.

34. What are records in C#?

Immutable reference types mainly for data models.

Example:



```
public record Person(string Name, int Age);
```

35. What is pattern matching in C#?

Simplifies conditional logic.

Example:

```
if (obj is string s && s.Length > 3) { ... }
```

36. Purpose of checked and unchecked

Control overflow behavior.

Example:

```
checked { int x = int.MaxValue + 1; } // exception
```

37. What is an anonymous type?

Type created on the fly.

Example:

```
var user = new { Name = "Ali", Age = 23 };
```

38. Purpose of nameof()

Returns variable or type name as string (safe).

Example:

```
Console.WriteLine(nameof(Person)); // "Person"
```

39. Difference between Task and Thread



- **Thread:** OS-level worker.
- **Task:** lightweight abstraction for async work.

40. Synchronous vs Asynchronous

- **Synchronous:** blocks until done.
- **Asynchronous:** runs without blocking.
Example:

```
await DownloadFileAsync();
```

41. What is the lock statement?

Prevents multiple threads accessing the same resource.

Example:

```
lock(obj) { counter++; }
```

42. What are lambda expressions?

Anonymous functions for short logic.

Example:

```
Func<int, int> square = x => x * x;
```

43. What is async and await?

Used for non-blocking async programming.

Example:

```
async Task LoadDataAsync() {
```



```
var data = await GetDataFromApi();  
}
```

44. Difference between IEnumerable and IEnumerator

- **IEnumerable:** returns IEnumerator.
- **IEnumerator:** moves through collection (`MoveNext`, `Current`).

45. Access modifiers: public, private, protected

- **public:** accessible anywhere
- **private:** within same class
- **protected:** within class + subclasses

46. Use of partial keyword

Split class or method definition across files.

Example:

```
partial class Person { void Part1() { } }  
partial class Person { void Part2() { } }
```

47. What are attributes in C#?

Metadata attached to code.

Example:

```
[Obsolete("Use NewMethod() instead")]  
void OldMethod() { }
```



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

48. What is reflection?

Inspect or modify code at runtime.

Example:

```
var props = typeof(Person).GetProperties();
```

49. What are nullable reference types?

Helps detect null reference issues at compile time.

Example:

```
string? name = null;
```

50. How does garbage collection work in .NET?

GC automatically frees memory of unreferenced objects.

Example:

```
person = null;  
GC.Collect(); // not usually needed manually
```

OOP Concepts

1. What are the four main principles of Object-Oriented Programming?

The four pillars of OOP are:



1. **Encapsulation** – hiding internal details of a class.
2. **Inheritance** – reusing code from another class.
3. **Polymorphism** – using the same interface with different implementations.
4. **Abstraction** – showing only essential features and hiding the rest.

Example:

Think of a **car** – you use the steering wheel (interface), but you don't need to know how the steering mechanism works internally (encapsulation and abstraction).

2. What is encapsulation?

Encapsulation means **bundling data and methods** that operate on that data into one unit (class), and restricting direct access from outside.

Example:

```
class BankAccount {  
    private decimal balance;  
  
    public void Deposit(decimal amount) => balance += amount;  
    public decimal GetBalance() => balance;  
}
```

Here, **balance** is **hidden** (private) — external code can't directly modify it.

3. What is inheritance?

Inheritance allows one class (child) to **reuse and extend** another class (parent).

Example:

```
class Vehicle {  
    public void Start() => Console.WriteLine("Starting...");  
}
```




```
}  
class Car : Vehicle {  
    public void Honk() => Console.WriteLine("Beep!");  
}
```

`Car` inherits from `Vehicle` — it gets `Start()` automatically.

4. What is polymorphism?

Polymorphism means “**many forms**” — the same method can behave differently depending on the object.

Example:

```
class Shape { public virtual void Draw() =>  
    Console.WriteLine("Drawing shape"); }  
class Circle : Shape { public override void Draw() =>  
    Console.WriteLine("Drawing circle"); }
```

```
Shape s = new Circle();  
s.Draw(); // "Drawing circle"
```

5. What is abstraction?

Abstraction is **showing only what's necessary** and hiding how it's done.

Example:

You can call `.SaveChanges()` on a `DbContext` in Entity Framework without knowing how it saves data to SQL — that's abstraction.

6. What is the difference between abstraction and encapsulation?

Concept	Purpose	Example
---------	---------	---------



Abstraction	Hides complex implementation	You use a car without knowing how the engine works
--------------------	------------------------------	--

Encapsulation	Restricts direct access to data	You can't touch the engine; you interact through methods
----------------------	---------------------------------	--

They often work **together** — abstraction defines “what to do,” encapsulation defines “how to hide it.”

7. What is a class in C#?

A **class** is a blueprint for creating objects — it defines properties, fields, and methods.

Example:

```
class Person {  
    public string Name;  
    public void Speak() => Console.WriteLine($"{Name} says hello!");  
}
```

8. What is an object in C#?

An **object** is an **instance** of a class — a real, usable version of it.

Example:

```
Person p = new Person();  
p.Name = "Alice";  
p.Speak(); // Alice says hello!
```

9. Difference between a class and an object

Class	Object
Blueprint	Instance of class



Doesn't take memory until instantiated Occupies memory

e.g., `Car`

e.g., `myCar = new
Car();`

10. What is a constructor in C#?

A **constructor** initializes a new object. It runs automatically when you create an instance.

Example:

```
class Car {  
    public string Model;  
    public Car(string model) { Model = model; }  
}  
Car c = new Car("Tesla");
```

11. What is a destructor in C#?

A **destructor** is called when an object is destroyed by the Garbage Collector to release resources.

Example:

```
~Car() { Console.WriteLine("Car destroyed"); }
```

12. Can a constructor be static?

Yes — a **static constructor** initializes static members of a class, runs only **once** automatically.

Example:

```
static Car() { Console.WriteLine("Static constructor called"); }
```

13. What is a static class?



A class that **cannot be instantiated** and contains only static members.

Example:

```
static class MathUtils {  
    public static int Add(int a, int b) => a + b;  
}
```

14. Difference between static and instance members

- **Static:** shared across all instances.
- **Instance:** unique per object.

Example:

```
class Employee {  
    public static int Count;  
    public string Name;  
}
```

15. What is method overloading?

Same method name, different parameters (compile-time polymorphism).

Example:

```
void Print(string msg) { }  
void Print(string msg, int times) { }
```

16. What is method overriding?

Redefining a base class method in a derived class using **override**.

Example:

```
class Animal { public virtual void Speak() =>  
    Console.WriteLine("Sound"); }
```



```
class Dog : Animal { public override void Speak() =>
Console.WriteLine("Bark"); }
```

17. Difference between overriding and overloading

Feature	Overloading	Overriding
When	Compile-time	Runtime
Parameter s	Must differ	Same
Keyword	None	<code>override</code>

18. What is a virtual method?

A method that can be **overridden** in a derived class.

Example:

```
public virtual void Speak() { ... }
```

19. What is the purpose of the **base** keyword?

Used to **call the parent class** constructor or method.

Example:

```
class Child : Parent {
    public Child() : base("value") { }
}
```

20. What is the **this** keyword used for?

Refers to the **current instance** of a class.

Example:



```
class Car {  
    string model;  
    public Car(string model) { this.model = model; }  
}
```

21. What is an abstract class?

A class that cannot be instantiated, used for defining **common base behavior**.

Example:

```
abstract class Shape {  
    public abstract void Draw();  
}
```

22. What is an interface?

An interface defines a **contract** — what methods or properties must exist.

Example:

```
interface IDrawable {  
    void Draw();  
}
```

23. Difference between abstract class and interface

Feature	Abstract Class	Interface
Can have implementation	Yes	No (until default methods in C# 8+)
Fields allowed	Yes	No
Multiple inheritance	No	Yes



24. Can an interface inherit another interface?

Yes, interfaces can inherit multiple interfaces:

```
interface IReadable { }  
interface IWritable { }  
interface IDocument : IReadable, IWritable { }
```

25. Can a class inherit multiple classes?

✗ No. C# does not support multiple class inheritance — only single inheritance.

26. Can a class implement multiple interfaces?

Yes.

```
class Printer : IReadable, IWritable { }
```

27. What happens if two interfaces have methods with the same name?

You can **explicitly implement** them:

```
void IReadable.Print() { ... }  
void IWritable.Print() { ... }
```

28. What is a sealed class?

A class that **cannot be inherited**.

Example:

```
sealed class Logger { }
```



29. What is a partial class?

A class split into multiple files.

Example:

```
// File1.cs
partial class Person { void Method1() { } }
// File2.cs
partial class Person { void Method2() { } }
```

30. What is a nested class?

A class declared **inside another class** for logical grouping.

Example:

```
class Outer {
    class Inner { }
}
```

31. Difference between a field and a property

Field	Property
Direct variable	Controlled access via getters/setters
No logic	Can include validation

32. What is an auto-implemented property?

A shorthand property with a hidden backing field.

Example:

```
public string Name { get; set; }
```



33. What is an indexer in OOP?

Allows object access like an array.

Example:

```
class Sample {  
    int[] data = new int[3];  
    public int this[int i] { get => data[i]; set => data[i] = value;  
}  
}
```

34. What are access modifiers in C#?

They define **who can access** members or classes (e.g., `public`, `private`, etc.).

35. What are the different access modifiers available in C#?

1. `public`
 2. `private`
 3. `protected`
 4. `internal`
 5. `protected internal`
 6. `private protected`
-

36. Difference between public, private, protected, and internal

Modifier	Accessible From
----------	--------------------



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

public	Anywhere
private	Same class
protected	Class + subclass
internal	Same assembly

37. What is multiple inheritance and is it supported in C#?

It means inheriting from multiple base classes — **not supported** in C#, but you can implement **multiple interfaces** to achieve similar behavior.

38. What are design patterns? Name a few commonly used ones.

Design patterns are **proven solutions to recurring design problems**.

Common patterns:

- Singleton
- Factory Method
- Observer
- Strategy
- Repository

Example: The **Singleton pattern** ensures only one instance of a class exists (like a global config manager).

39. What is dependency injection in OOP terms?

Dependency Injection (DI) means **passing dependencies** (objects) into a class instead of creating them inside.

Example:

Follow me on LinkedIn: <https://www.linkedin.com/in/sandeepall/>



```
class Report {  
    private ILogger _logger;  
    public Report(ILogger logger) { _logger = logger; }  
}
```

This improves testability and flexibility.

40. What are SOLID principles? Briefly explain each letter.

S – Single Responsibility

Each class should have only one job.

O – Open/Closed

Open for extension, closed for modification.

L – Liskov Substitution

Derived classes should work anywhere their base is expected.

I – Interface Segregation

Don't force classes to implement methods they don't need.

D – Dependency Inversion

Depend on abstractions, not concrete classes.

Example (real world):

In a payment system, a `PaymentProcessor` should depend on an `IPaymentGateway` interface — not a specific class like `PayPalGateway`.

.NET Core Basics

1. What is .NET Core?

.NET Core (now part of **.NET 5+ unified platform**) is a **cross-platform, open-source** framework for building applications on Windows, macOS, and Linux.

It's used for developing **web apps, APIs, console apps, and microservices**.

Example:

You can write a web API in .NET Core and deploy it on both **Windows Server** and **Linux Docker containers** without changing the code.



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

2. How is .NET Core different from .NET Framework?

Feature	.NET Framework	.NET Core / .NET 5+
Platform	Windows only	Cross-platform
Deployment	System-wide	Self-contained or portable
Open Source	No	Yes
Performance	Slower	Optimized and faster
Versioning	Global	Per-application

In short: .NET Core is modern, lightweight, modular, and **built for the cloud**.

3. What is the CLR (Common Language Runtime)?

The **CLR** is the heart of .NET — it **executes managed code**, provides **memory management**, **garbage collection**, **security**, and **exception handling**.

Think of it as the **.NET virtual machine**, similar to the JVM for Java.

4. What is the role of the SDK in .NET Core?

The **.NET SDK (Software Development Kit)** includes everything you need to **build and compile** .NET apps — the compiler, CLI tools, and libraries.

Example:

You need the SDK installed to run commands like:

```
dotnet new webapi
dotnet build
dotnet run
```

5. Difference between SDK, Runtime, and Hosting Bundle

Follow me on LinkedIn: <https://www.linkedin.com/in/sandeepall/>



Component	Purpose
SDK	Build and develop apps
Runtime	Run existing apps only
Hosting Bundle	Allows hosting ASP.NET Core apps on IIS

6. Main components of .NET Core architecture

1. **CLR (CoreCLR)** – executes code.
2. **BCL (Base Class Library)** – core functionality (collections, IO, etc.).
3. **CoreFX** – foundational libraries.
4. **Roslyn** – C# compiler.
5. **ASP.NET Core** – web framework on top of CoreCLR.

Think of it as **layers**: App → ASP.NET Core → CoreFX → CoreCLR.

7. What is the purpose of Program.cs in a .NET Core project?

It's the **entry point** of the application. It builds and runs the host.

Example:

```
var builder = WebApplication.CreateBuilder(args);  
var app = builder.Build();  
app.MapGet("/", () => "Hello World!");  
app.Run();
```

8. What is the Startup.cs file used for?



In older .NET Core projects (before minimal APIs), `Startup.cs` is where you **configure services and middleware**:

- `ConfigureServices()` → add dependencies
- `Configure()` → define HTTP pipeline

9. What is dependency injection (DI) in .NET Core?

It's a built-in **design pattern** for managing dependencies — instead of creating objects manually, the framework injects them.

Example:

```
builder.Services.AddScoped<IEmailService, EmailService>();
```

10. How do you register services in DI container?

You use one of three lifetimes:

```
services.AddSingleton<ILog, Logger>();  
services.AddScoped<IRepo, Repo>();  
services.AddTransient<IMail, MailService>();
```

Lifetime	Scope
Singleton	One instance for entire app
Scoped	One per request
Transient	New instance every time

11. What is middleware in .NET Core?

Middleware are **components that handle HTTP requests and responses** in a pipeline — each one can perform actions before/after passing control.



Example:

```
app.Use(async (context, next) => {  
    Console.WriteLine("Request started");  
    await next();  
    Console.WriteLine("Request ended");  
});
```

12. How do you configure middleware in the request pipeline?

Middleware is added in `Program.cs` (or `Startup.Configure()`):

```
app.UseRouting();  
app.UseAuthentication();  
app.UseAuthorization();  
app.MapControllers();
```

Order **matters** — requests pass through them sequentially.

13. Difference between Use, Run, and Map

Method	Description
Use	Adds middleware and passes control to next
Run	Terminates pipeline (no next)
Map	Branches pipeline based on path

Example:

```
app.Map("/admin", adminApp => adminApp.Run(async ctx => await  
ctx.Response.WriteAsync("Admin")));
```



14. What is Kestrel in .NET Core?

Kestrel is the **default built-in web server** for ASP.NET Core — it's **lightweight, high-performance, and cross-platform**.

15. Can you host .NET Core applications on IIS?

Yes.

You can host on IIS using the **ASP.NET Core Hosting Bundle**, which proxies requests from IIS to Kestrel.

16. Difference between self-hosting and IIS hosting

Type	Description
Self-hosted	App runs independently (Kestrel directly serves)
IIS-hosted	IIS forwards requests to Kestrel

Self-hosting is common in Docker or Linux environments.

17. What are configuration providers in .NET Core?

They load configuration from multiple sources:

- appsettings.json
- environment variables
- command-line args
- Azure Key Vault
- User secrets

Example:



```
builder.Configuration.AddJsonFile("appsettings.json")
                        .AddEnvironmentVariables();
```

18. How does .NET Core handle configuration files?

All configuration data is read into the **IConfiguration** object, which supports hierarchical data (like JSON).

19. What is the appsettings.json file used for?

Stores application configuration (e.g., connection strings, API keys):

```
{
  "ConnectionStrings": {
    "Default": "Server=.;Database=Shop;Trusted_Connection=True;"
  }
}
```

20. How can you use environment-based configuration?

Use files like `appsettings.Development.json` or `appsettings.Production.json`.
The environment is set using:

```
set ASPNETCORE_ENVIRONMENT=Development
```

21. What is the IConfiguration interface used for?

Used to **access configuration values**:

```
var conn = Configuration["ConnectionStrings:Default"];
```



22. What is the purpose of IOptions pattern in configuration management?

It binds configuration sections to strongly typed objects.

```
services.Configure<MyConfig>(builder.Configuration.GetSection("MyConfig"));
```

23. How do you set up logging in .NET Core?

Logging is built-in via `ILogger<T>` and configured in `Program.cs`:

```
builder.Logging.AddConsole().AddDebug();
```

24. What logging providers are available in .NET Core?

- Console
 - Debug
 - EventSource
 - Azure App Insights
 - Serilog / NLog (3rd party)
-

25. What are different project types in .NET Core?

- Console App
- Web App (MVC/Razor)
- Web API



- Worker Service
- Class Library
- Blazor App

Example:

```
dotnet new webapi -n MyApi
```

26. Difference between net6.0, net7.0, and net8.0 target frameworks

Version	Release	Highlights
.NET 6	LTS	Minimal APIs, cross-platform
.NET 7	STS	Performance & cloud-native improvements
.NET 8	LTS	AI & Native AOT, better performance

27. How do you create a new .NET Core project using CLI?

```
dotnet new console -n MyApp  
dotnet new webapi -n MyApi
```

28. What command is used to run a .NET Core application?

```
dotnet run
```

29. What is the role of launchSettings.json?

It stores environment settings used when running locally — e.g., profiles, ports, and environment name.

Example:



```
"profiles": {
  "MyApp": {
    "commandName": "Project",
    "launchUrl": "weatherforecast",
    "environmentVariables": {
      "ASPNETCORE_ENVIRONMENT": "Development"
    }
  }
}
```

30. Difference between build, publish, and run commands

Command	Purpose
build	Compiles the project
publish	Prepares app for deployment
run	Builds and runs locally

31. How does DI differ from Service Locator pattern?

- **DI:** dependencies are provided via constructor.
- **Service Locator:** classes fetch dependencies from a central locator (harder to test, less transparent).

DI is **preferred** in .NET Core.

32. What is cross-platform development in .NET Core?

The ability to run .NET apps on **Windows, macOS, and Linux** using the same codebase — perfect for Docker or cloud apps.



33. What are runtime identifiers (RIDs) in .NET Core?

RIDs specify the **target OS and architecture** for builds.

Examples:

- `win-x64`
- `linux-x64`
- `osx-arm64`

Used in `runtimeconfig.json` or `dotnet publish`.

34. Difference between framework-dependent and self-contained deployments

Type	Description
Framework-dependent	Relies on shared .NET runtime
Self-contained	Includes its own runtime (larger size, runs anywhere)

Command:

```
dotnet publish -r win-x64 --self-contained true
```

35. How do you handle exceptions globally in .NET Core?

Use **Exception Handling Middleware** or `UseExceptionHandler`:

```
app.UseExceptionHandler("/Home/Error");
```

You can also use filters (`IExceptionHandler`) in MVC.



36. What is UseExceptionHandler middleware used for?

It provides a **centralized error page or JSON response** for unhandled exceptions.

37. What is the dotnet restore command used for?

It restores NuGet packages required by the project.

Example:

```
dotnet restore
```

38. Difference between NuGet packages and project references

Type	Description
NuGet Package	External library from NuGet.org
Project Reference	Reference to another local project in solution

39. Benefits of using .NET Core for web development

Cross-platform

Built-in dependency injection

High performance with Kestrel

Cloud-ready (Docker, Azure)

Open-source and modular

40. New features introduced in .NET 6 / 7 / 8

- **Minimal APIs** (smaller, faster endpoints)
- **Hot Reload**
- **Native AOT** (ahead-of-time compilation)



- Improved LINQ performance
- Enhanced MAUI for mobile apps
- Span<T>, record structs, and better memory management

ASP.NET MVC / Web API

1. What is ASP.NET MVC?

ASP.NET MVC (Model-View-Controller) is a **web application framework** from Microsoft that separates an app into **three main components**:

- **Model** (data & logic)
- **View** (UI)
- **Controller** (request handling)

Example:

When you visit `/Products/Details/5` →

Controller = `ProductsController`, Action = `Details()`, Parameter = `5`.

2. What are the main components of MVC architecture?

1. **Model** – represents the data and business logic.
2. **View** – defines how data is displayed.
3. **Controller** – handles user input and coordinates between Model and View.

3. What is the role of a Controller in MVC?



The **controller** handles **HTTP requests**, interacts with the **model**, and returns a **view or result**.

Example:

```
public class HomeController : Controller {
    public IActionResult Index() {
        var products = _repo.GetAll();
        return View(products);
    }
}
```

4. What is a View in MVC?

The **View** is the **presentation layer** — it displays data returned by the controller, usually in HTML via Razor syntax.

Example:

```
@model IEnumerable<Product>
@foreach (var p in Model) {
    <p>@p.Name - @p.Price</p>
}
```

5. What is a Model in MVC?

The **Model** contains the **data and business logic** of the app.
It's often a class that maps to a database entity.

Example:

```
public class Product {
    public int Id { get; set; }
    public string Name { get; set; }
}
```



6. Difference between MVC and Web Forms

Feature	ASP.NET MVC	ASP.NET Web Forms
Architecture	MVC pattern	Event-driven
Testability	Easy (separation of concerns)	Harder
HTML Control	Full control	Server controls
ViewState	No	Yes
URL Routing	Clean URLs	Postbacks & querystrings

7. What is routing in ASP.NET MVC?

Routing maps **URLs to controller actions**.

Example:

`/Products/Details/5` → `ProductsController.Details(5)`

8. What is the default route pattern in MVC?

`{controller=Home}/{action=Index}/{id?}`

If you don't specify, MVC uses **HomeController** → **Index()** by default.

9. How do you define a custom route in MVC?

Example:

```
routes.MapRoute(  
    name: "ProductRoute",  
    template: "store/{category}/{id}",  
    defaults: new { controller = "Product", action = "Category" }  
);
```



10. What is attribute routing?

Instead of defining routes globally, you decorate actions or controllers with `[Route]` attributes.

Example:

```
[Route("api/products/{id}")]  
public IActionResult GetProduct(int id) { ... }
```

11. Difference between conventional and attribute routing

Type	Defined Where	Example
Conventional	Startup route config	<code>"{controller}/{action}/{id?}"</code>
Attribute	Directly on controller/action	<code>[Route("api/items/{id}")]</code>

12. What is the purpose of filters in MVC?

Filters let you **run code before or after** certain stages of request processing — like authorization, caching, or logging.

13. What are the types of filters available in MVC?

1. **Authorization filters** – security
2. **Action filters** – before/after action executes
3. **Result filters** – before/after view renders
4. **Exception filters** – handle errors
5. **Resource filters** – manage caching/resources



14. What is an Action Filter?

Executes custom logic **before or after** a controller action runs.

Example:

```
public class LogActionFilter : ActionFilterAttribute {  
    public override void OnActionExecuting(ActionExecutingContext  
context) {  
        Console.WriteLine("Action is starting");  
    }  
}
```

15. What is the difference between ActionResult and IActionResult?

- **ActionResult**: base class for MVC-specific results.
- **IActionResult**: interface implemented by both MVC and Web API results.

Prefer **IActionResult** in ASP.NET Core for flexibility.

16. What is the use of ViewBag, ViewData, and TempData?

Object	Lifetime	Type	Purpose
ViewData	Single request	Dictionary	Pass data from Controller → View
ViewBag	Single request	Dynamic	Simplified ViewData
TempData	Multiple requests	Dictionary	Stores data between redirects

Example:

```
ViewBag.Message = "Welcome!";  
TempData["Notice"] = "Saved successfully";
```




17. Difference between ViewBag and TempData

- **ViewBag** works only for the **current request**.
- **TempData** persists **until the next request** (like after **RedirectToAction()**).

18. How do you return JSON data from a controller?

Example:

```
public JsonResult GetProducts() {  
    var data = _repo.GetAll();  
    return Json(data);  
}
```

In ASP.NET Core:

```
return new JsonResult(products);
```

19. How do you handle form submission in MVC?

1. Create a form in the view:

```
<form asp-action="Submit" method="post">  
    <input asp-for="Name" />  
    <button type="submit">Save</button>  
</form>
```

2. Handle it in the controller:

```
[HttpPost]
```




```
public IActionResult Submit(Product model) { ... }
```

20. What is model binding in MVC?

Model binding automatically maps **form inputs or query parameters** to C# objects.

Example:

A form field named `Name` binds to a `Product.Name` property.

21. What is model validation?

ASP.NET automatically validates incoming data based on data annotations or custom logic.

22. How do you apply validation in MVC?

Add **data annotation attributes** to model properties:

```
public class Product {  
    [Required]  
    [Range(1, 100)]  
    public decimal Price { get; set; }  
}
```

Then check:

```
if (ModelState.IsValid) { ... }
```

23. What are data annotation attributes?

Attributes used for validation and display purposes:

- `[Required]`



- `[StringLength(50)]`
- `[Range(1, 10)]`
- `[EmailAddress]`

24. Purpose of `HttpGet` and `HttpPost` attributes

- `[HttpGet]`: used for **retrieving data** (default).
- `[HttpPost]`: used for **sending data** (form submissions).

25. Difference between `RedirectToAction()` and `View()`

Method	Description
<code>View()</code>	Renders a View directly
<code>RedirectToAction()</code>	Redirects to another action (new request)

26. How do you handle errors in MVC?

Use:

- `try-catch` blocks
- `ExceptionHandler` middleware
- `HandleError` filter
- Custom error pages



27. What is the HandleError attribute used for?

Automatically catches exceptions and shows a custom error view.

```
[HandleError(View = "ErrorPage")]
public IActionResult Index() { ... }
```

28. Difference between Html.Partial() and Html.RenderPartial()

Method	Returns	Performance
<code>Html.Partial()</code>	Returns HTML string	Slightly slower
<code>Html.RenderPartial()</code>	Writes directly to response	Faster

29. What is a layout page in MVC?

It's like a **master page** that defines common structure (e.g., header, footer).

Example:

`_Layout.cshtml` used by multiple views via:

```
@{
    Layout = "_Layout";
}
```

30. What is bundling and minification?

- **Bundling:** combines multiple CSS/JS files.
- **Minification:** removes spaces/comments to reduce file size.

Improves **page load performance**.



31. What are Razor views?

Razor is a **templating engine** that mixes C# code with HTML using @ syntax.

Example:

```
<h1>Hello @Model.Name</h1>
```

32. Difference between Razor and ASPX view engines

Feature	Razor	ASPX
Syntax	@	<% %>
Lightweight	Yes	No
File Extension	.cshtml	.aspx

33. What is an API Controller in ASP.NET Core Web API?

It handles **HTTP API requests** and returns data (usually JSON), not HTML views.

Example:

```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase {
    [HttpGet]
    public IEnumerable<Product> Get() => _repo.GetAll();
}
```

34. Difference between MVC Controller and API Controller

Feature	MVC Controller	API Controller
---------	----------------	----------------



Returns	Views (HTML)	Data (JSON/XML)
Base Class	Controller	ControllerBase
Use Case	Web apps	REST APIs

35. What is REST in Web API?

REST (Representational State Transfer) is an **architecture style** that uses standard HTTP methods (GET, POST, PUT, DELETE) for CRUD operations.

36. What are the HTTP methods supported by Web API?

- **GET** – Retrieve data
 - **POST** – Create data
 - **PUT** – Update data
 - **DELETE** – Remove data
 - **PATCH** – Partial update
-

37. What is the use of [FromBody] and [FromQuery] attributes?

They specify **where** to read parameters from.
Example:

```
[HttpPost]
public IActionResult Add([FromBody] Product product)

[HttpGet]
public IActionResult Search([FromQuery] string keyword)
```



38. How do you enable CORS in Web API?

Add it in `Program.cs`:

```
builder.Services.AddCors();  
app.UseCors(p =>  
p.AllowAnyOrigin().AllowAnyMethod().AllowAnyHeader());
```

39. How do you version APIs in ASP.NET Core?

Use **API Versioning Package**:

```
dotnet add package Microsoft.AspNetCore.Mvc.Versioning
```

Configure:

```
options.AssumeDefaultVersionWhenUnspecified = true;  
options.DefaultApiVersion = new ApiVersion(1, 0);
```

40. Difference between synchronous and asynchronous Web API methods

Type	Behavior
Synchronous	Blocks thread until operation finishes
Asynchronous	Frees thread to handle other requests (<code>async/await</code>)

Example:

```
public async Task<IActionResult> GetProductsAsync() =>  
    Ok(await _repo.GetProductsAsync());
```




Entity Framework / LINQ

1. What is Entity Framework (EF)?

Entity Framework is an **Object-Relational Mapper (ORM)** for .NET.

It allows you to interact with a **database using C# classes** instead of raw SQL. EF handles **CRUD operations, relationships, and change tracking** automatically.

Example:

```
var products = dbContext.Products.ToList();
```

This fetches all products from the database without writing SQL.

2. Advantages of using Entity Framework

- Eliminates most manual SQL writing
 - Supports **LINQ queries**
 - Automatic **change tracking**
 - Works with **multiple databases**
 - Supports **migrations** for schema changes
 - Improves maintainability of data layer
-

3. Different approaches to use EF

1. **Code First** – Create classes first; database is generated.
2. **Database First** – Use an existing database; EF generates classes.



3. **Model First** – Design a model in Visual Studio; EF generates database & classes.

4. What is Code First in EF?

You define **C# classes (models)** and EF generates the database.

Example:

```
public class Product {  
    public int Id { get; set; }  
    public string Name { get; set; }  
}
```

5. What is Database First in EF?

You start with an **existing database**, and EF scaffolds **classes** and **DbContext** for you.

6. What is Model First in EF?

You design an **EDMX model** in Visual Studio and EF generates **database tables and classes** from it.

7. What is an Entity in EF?

An **entity** is a **class** that maps to a **database table**.

Example: **Product** is an entity corresponding to the **Products** table.

8. What is a DbContext class?

DbContext is the **main EF class** for interacting with the database.

It manages entities, change tracking, and saving data.

Example:



```
public class AppDbContext : DbContext {  
    public DbSet<Product> Products { get; set; }  
}
```

9. What is DbSet in EF?

A `DbSet<TEntity>` represents a **table in the database** and allows **querying and CRUD operations**.

10. Purpose of OnModelCreating method

`OnModelCreating` allows **Fluent API configurations** for tables, relationships, keys, etc.

Example:

```
protected override void OnModelCreating(ModelBuilder modelBuilder) {  
    modelBuilder.Entity<Product>().HasKey(p => p.Id);  
}
```

11. What is a migration in EF Core?

Migration represents a **schema change** in the database corresponding to your **C# models**.

12. How do you add a migration in EF Core?

```
dotnet ef migrations add InitialCreate
```

13. Command to update database using migrations

```
dotnet ef database update
```



14. What is lazy loading in EF?

EF loads related entities only when accessed.

Example:

```
var order = dbContext.Orders.Find(1);  
var products = order.Products; // Loaded on access
```

15. What is eager loading in EF?

EF loads related entities immediately using `Include()`.

Example:

```
var orders = dbContext.Orders.Include(o => o.Products).ToList();
```

16. What is explicit loading in EF?

Explicitly load related entities when needed:

```
var order = dbContext.Orders.Find(1);  
dbContext.Entry(order).Collection(o => o.Products).Load();
```

17. Difference between lazy and eager loading

Type	When loaded	Query executed
Lazy	On access	Separate query
Eager	Immediately	Single query with <code>Include()</code>

18. How to enable lazy loading in EF Core

Install `Microsoft.EntityFrameworkCore.Proxies` and configure:



```
options.UseLazyLoadingProxies();
```

Also, navigation properties must be **virtual**.

19. Difference between tracked and no-tracking queries

- **Tracked:** EF monitors changes to update DB (`dbContext.Products.ToList()`)
 - **No-tracking:** Read-only queries, better performance (`dbContext.Products.AsNoTracking().ToList()`)
-

20. What is a navigation property?

Property that **links entities** in relationships (FK).

Example:

```
public class Order {  
    public int Id { get; set; }  
    public virtual List<Product> Products { get; set; }  
}
```

21. Difference between one-to-many and many-to-many

- **One-to-many:** One entity → multiple related entities
 - **Many-to-many:** Multiple entities → multiple related entities (via join table)
-

22. How to configure relationships in EF Core

- **Data Annotations**



```
public int CustomerId { get; set; }  
[ForeignKey("CustomerId")]  
public Customer Customer { get; set; }
```

- **Fluent API**

```
modelBuilder.Entity<Order>()  
    .HasOne(o => o.Customer)  
    .WithMany(c => c.Orders)  
    .HasForeignKey(o => o.CustomerId);
```

23. What are data annotations in EF Core?

Attributes that configure entity properties:

- `[Key]` – primary key
- `[Required]` – not null
- `[MaxLength(50)]` – string length

24. What is Fluent API in EF Core?

A **programmatic way** to configure entities instead of data annotations.

Example:

```
modelBuilder.Entity<Product>().Property(p => p.Name).IsRequired();
```

25. What is a primary key and how is it defined in EF?

The **unique identifier** for an entity:

`[Key]`



```
public int Id { get; set; }
```

Or Fluent API:

```
modelBuilder.Entity<Product>().HasKey(p => p.Id);
```

26. How to handle concurrency in EF Core

Use **row versioning**:

```
[Timestamp]  
public byte[] RowVersion { get; set; }
```

27. Difference between Add, Update, and Attach

- **Add** – marks as new entity
 - **Update** – marks as modified
 - **Attach** – tracks without marking as new/modified
-

28. Difference between SaveChanges() and SaveChangesAsync()

- `SaveChanges()` – synchronous DB save
 - `SaveChangesAsync()` – asynchronous save, improves scalability
-

29. What are shadow properties in EF Core?

Properties **not in C# class** but tracked by EF in DB. Useful for audit fields.



30. What is change tracking in EF Core?

EF keeps track of **entity states** (Added, Modified, Deleted) to update DB efficiently.

31. What is a stored procedure and can it be used in EF?

A stored procedure is **predefined SQL** in DB. EF Core can call them:

```
var products = dbContext.Products.FromSqlRaw("EXEC  
GetProducts").ToList();
```

32. What is LINQ in C#?

LINQ = **Language Integrated Query**, allows **querying objects, collections, or databases** using C# syntax.

33. Difference between LINQ to Objects, LINQ to Entities, LINQ to SQL

Type	Query Target	Notes
LINQ to Objects	In-memory collections	e.g., List<Product>
LINQ to Entities	EF entities	Translates to SQL
LINQ to SQL	SQL Server only	Legacy ORM

34. Extension methods in LINQ

LINQ uses **extension methods** to extend collections:

```
var cheapProducts = products.Where(p => p.Price < 100).ToList();
```

35. Difference between Select and SelectMany



- **Select** – projects each element
- **SelectMany** – flattens collections of collections

Example:

```
Select: [[1,2],[3,4]] => [[1,2],[3,4]]
```

```
SelectMany: [[1,2],[3,4]] => [1,2,3,4]
```

36. Difference between First(), FirstOrDefault(), Single(), SingleOrDefault()

Method	Throws Exception	Default Value if empty
First()	Yes	No default
FirstOrDefault()	No	default(T)
Single()	Yes if >1	No default
SingleOrDefault()	Yes if >1	default(T)

37. What does Any() do in LINQ?

Checks if **any elements satisfy a condition**:

```
bool exists = products.Any(p => p.Price > 100);
```

38. How do you perform joins using LINQ?

```
var result = from o in orders
              join c in customers on o.CustomerId equals c.Id
              select new { o.Id, c.Name };
```



39. What is deferred execution in LINQ?

Queries aren't executed until you iterate over them (`ToList()`, `foreach`).

Example:

```
var query = dbContext.Products.Where(p => p.Price > 100); // not executed
var list = query.ToList(); // executed now
```

40. How do you debug LINQ queries in EF Core?

- Use `ToQueryString()` in EF Core:

```
Console.WriteLine(dbContext.Products.Where(p => p.Price > 100).ToQueryString());
```

- Log SQL using `ILogger` in DbContext options.

SQL / Database

1. What is a primary key in a database?

A **primary key** uniquely identifies each row in a table.

- Must be **unique** and **not null**.
- A table can have **only one primary key**.

Example:

```
CREATE TABLE Customers (  
    CustomerID INT PRIMARY KEY,  
    Name NVARCHAR(50)  
);
```



2. What is a foreign key?

A **foreign key** is a column that **links to the primary key of another table**, enforcing referential integrity.

Example:

```
CREATE TABLE Orders (  
    OrderID INT PRIMARY KEY,  
    CustomerID INT,  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);
```

3. Difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN

Join Type	Result
INNER JOIN	Only matching rows from both tables
LEFT JOIN	All rows from left table + matching rows from right table
RIGHT JOIN	All rows from right table + matching rows from left table
FULL OUTER JOIN	All rows from both tables, NULL if no match

Example:

```
SELECT Customers.Name, Orders.OrderID  
FROM Customers  
LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

4. What is normalization? Explain the normal forms

Normalization organizes tables to **reduce redundancy**:



1. **1NF (First Normal Form)** – No repeating groups, atomic values only
 2. **2NF (Second Normal Form)** – 1NF + every non-key column fully depends on primary key
 3. **3NF (Third Normal Form)** – 2NF + no transitive dependency
 4. **BCNF (Boyce-Codd Normal Form)** – Every determinant is a candidate key
 5. **4NF & 5NF** – Handle multi-valued dependencies and join dependencies
-

5. What is denormalization?

Denormalization **adds redundancy** to improve performance (fewer joins).

- Example: storing **CustomerName** in **Orders** table for faster reads.
-

6. What is a unique key?

A **unique key** ensures **no duplicate values** in a column.

- Can be **null** in some databases
 - A table can have **multiple unique keys**
-

7. What is an index?

An **index** improves query performance by allowing faster lookup.

Example:

```
CREATE INDEX idx_name ON Customers(Name);
```



8. Clustered vs Non-clustered indexes

Type	Description
Clustered	Sorts and stores table rows physically (1 per table)
Non-clustered	Separate structure pointing to table rows (multiple per table)

9. Difference between DELETE and TRUNCATE

Command	Rows Removed	Logging	Can use WHERE	Identity reset
DELETE	Specified	Fully	Yes	No
TRUNCATE	All rows	Minimal	No	Yes

10. Difference between CHAR and VARCHAR

Type	Description
CHAR(n)	Fixed-length, pads extra spaces
VARCHAR(n)	Variable-length, stores only used space

11. Difference between NVARCHAR and VARCHAR

- **NVARCHAR** stores **Unicode characters** (multi-language)
- **VARCHAR** stores **non-Unicode characters** (ASCII)

12. What is a stored procedure?

A **precompiled SQL block** stored in DB that can be **reused**.

Example:



```
CREATE PROCEDURE GetCustomerOrders @CustomerID INT
AS
SELECT * FROM Orders WHERE CustomerID = @CustomerID;
```

13. How do you call a stored procedure in SQL Server?

```
EXEC GetCustomerOrders @CustomerID = 1;
```

14. What is a trigger in SQL?

A **trigger** is a SQL procedure that automatically executes on **INSERT, UPDATE, or DELETE**.

Example:

```
CREATE TRIGGER trgAfterInsert
ON Orders
AFTER INSERT
AS
PRINT 'Order added!';
```

15. Difference between WHERE and HAVING

- **WHERE** filters rows **before** aggregation
- **HAVING** filters rows **after** aggregation

Example:

```
SELECT CustomerID, COUNT(*) AS OrdersCount
FROM Orders
GROUP BY CustomerID
HAVING COUNT(*) > 5;
```



16. Difference between UNION and UNION ALL

- **UNION** – removes duplicates
 - **UNION ALL** – keeps duplicates
-

17. What is a view in SQL?

A **virtual table** based on a query. Does **not store data physically**.

Example:

```
CREATE VIEW CustomerOrders AS  
SELECT Customers.Name, Orders.OrderID  
FROM Customers  
JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

18. What is a materialized view?

A **physical copy** of the query result stored in DB for faster access.

- Needs to be **refreshed** when data changes.
-

19. What is a transaction?

A **unit of work** in SQL that is **atomic** — either all succeed or all fail.

20. ACID properties of a transaction

- **Atomicity** – All or nothing
- **Consistency** – DB remains consistent



- **Isolation** – Transactions don't interfere
 - **Durability** – Changes persist after commit
-

21. Difference between COMMIT and ROLLBACK

- **COMMIT** – saves changes
 - **ROLLBACK** – reverts changes
-

22. What is a cursor in SQL?

A cursor allows **row-by-row processing** of query results.

Example:

```
DECLARE customer_cursor CURSOR FOR  
SELECT Name FROM Customers;
```

23. Common aggregate functions

- **COUNT()** – number of rows
 - **SUM()** – total of a column
 - **AVG()** – average value
 - **MIN()** / **MAX()** – min/max values
-

24. What is a subquery?



A query inside another query.

Example:

```
SELECT Name FROM Customers
WHERE CustomerID IN (SELECT CustomerID FROM Orders WHERE OrderDate >
'2025-01-01');
```

25. What is a correlated subquery?

A subquery that **depends on the outer query**.

Example:

```
SELECT Name FROM Customers c
WHERE EXISTS (SELECT 1 FROM Orders o WHERE o.CustomerID =
c.CustomerID);
```

26. Difference between INNER JOIN and CROSS JOIN

- **INNER JOIN** – matches rows based on condition
 - **CROSS JOIN** – Cartesian product (all combinations)
-

27. Difference between DELETE, DROP, and TRUNCATE

Command	Description
DELETE	Remove rows (can filter with WHERE)
TRUNCATE	Remove all rows, fast
DROP	Delete table/DB structure permanently

28. Difference between CHAR and TEXT



- **CHAR** – fixed-length small strings
- **TEXT** – stores large variable-length text

29. How to prevent SQL injection

- Use **parameterized queries** or **stored procedures**

Example:

```
var command = new SqlCommand("SELECT * FROM Users WHERE Name=@name",  
conn);  
command.Parameters.AddWithValue("@name", userInput);
```

30. What are stored functions?

SQL functions that **return a value**. Unlike stored procedures, can be used in SELECT statements.

31. What is a schema in SQL Server?

A **namespace/container** for tables, views, procedures to organize DB objects.

Example: **dbo.Customers** – **dbo** is schema.

32. Difference between PRIMARY KEY and FOREIGN KEY

Key	Purpose
PRIMARY KEY	Uniquely identifies a row
FOREIGN KEY	Enforces referential integrity



33. Difference between IDENTITY and SEQUENCE

- **IDENTITY** – auto-increment column tied to a table
 - **SEQUENCE** – independent number generator
-

34. What is indexing and why important?

Indexes **speed up queries** by allowing fast row lookup.

- Can be clustered or non-clustered
 - Improves SELECT, slows down INSERT/UPDATE slightly
-

35. Difference between clustered index and primary key

- Primary key is **logical constraint**
 - Clustered index is **physical storage order**
 - Usually, PK **creates clustered index by default**
-

36. What is a deadlock in SQL?

A situation where two or more transactions block each other, unable to proceed.

37. What is a composite key?

A primary key with multiple columns.

Example:

PRIMARY KEY (OrderID, ProductID)



38. Transaction isolation levels

- **READ UNCOMMITTED** – dirty reads allowed
- **READ COMMITTED** – no dirty reads
- **REPEATABLE READ** – no non-repeatable reads
- **SERIALIZABLE** – strictest, no phantom reads

39. Difference between INNER JOIN and SELF JOIN

- **INNER JOIN** – join between different tables
- **SELF JOIN** – join **table with itself**

Example:

```
SELECT e1.Name AS Employee, e2.Name AS Manager
FROM Employees e1
JOIN Employees e2 ON e1.ManagerID = e2.EmployeeID;
```

40. How to optimize SQL queries

- Use proper **indexes**
- Avoid **SELECT ***
- Use **joins efficiently**
- Filter early with **WHERE**



- Use stored procedures
- Analyze query execution plan

REST / Microservices

1. What is REST?

REST (**R**epresentational **S**tate **T**ransfer) is an **architectural style** for designing web services that **use standard HTTP methods** and work with **resources**.

- Focuses on **simplicity, scalability, and stateless communication**.

Example: <https://api.example.com/customers/1> represents a **customer resource**.

2. Key principles of REST architecture

1. **Client-Server**: Separation of concerns (client handles UI, server handles data).
2. **Stateless**: Each request contains all necessary info; no server session.
3. **Cacheable**: Responses can be cached to improve performance.
4. **Uniform Interface**: Standardized way to interact with resources.
5. **Layered System**: Clients don't care if they are connected to intermediary servers.
6. **Code on Demand (optional)**: Server can send executable code to client.

3. Difference between REST and SOAP



Aspect	REST	SOAP
Protocol	HTTP	Protocol-specific (SOAP over HTTP, SMTP, etc.)
Format	JSON, XML, others	XML only
State	Stateless	Can be stateful
Flexibility	High	Low (strict standards)
Performance	Lightweight	Heavier due to XML

4. What is a RESTful API?

A **RESTful API** is a web API that **follows REST principles**, exposing **resources** via URLs and using **HTTP methods** to operate on them.

Example:

```
GET /customers/1
POST /orders
PUT /orders/1
DELETE /orders/1
```

5. Common HTTP methods in REST APIs

Method	Purpose
GET	Retrieve resource
POST	Create resource
PUT	Update resource (replace)
PATCH	Update resource (partial)
DELETE	Delete resource



6. Difference between GET, POST, PUT, PATCH, DELETE

- **GET** – Read-only, safe, idempotent
- **POST** – Create new resource, not idempotent
- **PUT** – Replace entire resource, idempotent
- **PATCH** – Partially update resource
- **DELETE** – Remove resource, idempotent

7. What is statelessness in REST?

Each request must **contain all info** needed by the server.

- Server does not store session info.
Benefit: Scalability and reliability.

8. Purpose of status codes in REST APIs

HTTP status codes **indicate result** of a request. Helps client understand **success, failure, or action needed**.

9. Common HTTP status codes

Code	Meaning
200	OK – Success
201	Created – Resource created
204	No Content – Success, no response body
400	Bad Request – Client error



- 401 Unauthorized – Authentication required
 - 403 Forbidden – Authenticated but not allowed
 - 404 Not Found – Resource missing
 - 500 Internal Server Error – Server error
-

10. What is HATEOAS?

HATEOAS (**H**ypermedia as the **E**ngine of **A**pplication **S**tate) allows clients to **navigate resources dynamically** using hyperlinks provided by the API.

Example:

```
{
  "customerId": 1,
  "name": "John",
  "links": {
    "orders": "/customers/1/orders",
    "self": "/customers/1"
  }
}
```

11. How do you version a REST API?

- **URL versioning:** `/api/v1/customers`
 - **Header versioning:** `Accept: application/vnd.example.v1+json`
 - **Query string versioning:** `/api/customers?version=1`
-

12. What is JWT (JSON Web Token)?

JWT is a **compact token format** used for **authentication and authorization**.



- Contains header, payload, signature
- Stateless; can be verified without DB lookup

Example:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

13. Authentication vs Authorization

Aspect	Authentication	Authorization
Purpose	Verify identity	Determine access rights
Example	Login with username/password	Check if user can delete a resource

14. What is rate limiting in APIs?

Restricting **number of requests per client** in a given time to **prevent abuse**.

Example: 1000 requests per hour.

15. Logging in microservices

- Use centralized logging tools: **ELK Stack, Prometheus, Grafana**
- Helps **debug and monitor** distributed services

16. Monolithic vs Microservice architecture

Aspect	Monolithic	Microservices
Deployment	Single unit	Independent services



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

Scalability	Harder	Easier, per service
Development	Centralized	Decentralized
Failure impact	All system affected	Isolated

17. Benefits of microservices

- Independent deployment
 - Scalability per service
 - Technology flexibility
 - Fault isolation
-

18. Challenges of microservices

- Complexity in communication
 - Data consistency
 - Distributed transactions
 - Monitoring and logging
-

19. What is service discovery?

Mechanism to **find service instances dynamically** in microservices.

Tools: **Eureka, Consul, Zookeeper**

20. What is an API gateway?

Acts as a **single entry point** for multiple microservices.



- Handles **routing, authentication, rate limiting, caching.**

21. How microservices communicate

- **Synchronous:** HTTP/REST, gRPC
- **Asynchronous:** Message queues (RabbitMQ, Kafka)

22. Difference between synchronous and asynchronous communication

Type	Description	Example
Synchronous	Client waits for response	REST API call
Asynchronous	Client doesn't wait	Kafka message queue

23. Purpose of message queues

- Decouple services
- Handle high traffic
- Ensure **reliability and eventual consistency**

24. What is eventual consistency?

- Data may be temporarily inconsistent across services but **will become consistent eventually.**
- Common in distributed systems.



25. Circuit breaker pattern

Prevents **repeated calls to a failing service**, improving system stability.

Tools: **Polly (C#), Hystrix (Java)**

26. Containerization vs Virtualization

Aspect	Virtualization	Containerization
OS	Full guest OS	Shares host OS
Overhead	High	Low
Startup	Minutes	Seconds
Isolation	Strong	Moderate

27. Role of Docker in microservices

- Packages **microservices and dependencies** into containers
 - Ensures **consistent environments across dev, test, production**
 - Lightweight and portable
-

28. How to handle versioning in microservices

- **URL versioning:** `/v1/users`
 - **Header versioning:** `X-API-Version: 1`
 - Maintain backward compatibility
-



29. Common monitoring tools

- **Prometheus + Grafana** – Metrics and dashboards
- **ELK Stack** – Centralized logs
- **Jaeger / Zipkin** – Distributed tracing
- **New Relic / Datadog** – Performance monitoring

Common Tools & Best Practices

Software Design Principles, Testing, and Best Practices — Q&A

1. What is SOLID principle? Briefly explain each principle.

SOLID is a set of five principles for object-oriented programming that help write maintainable, scalable, and flexible code.

1. **S – Single Responsibility Principle (SRP)**

- A class should have **only one reason to change**.
Example: `InvoiceProcessor` handles invoices, not email notifications.

2. **O – Open/Closed Principle (OCP)**

- Classes should be **open for extension but closed for modification**.
Example: Use inheritance or interfaces to add new functionality without changing existing code.

3. **L – Liskov Substitution Principle (LSP)**



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

- Subtypes should **replace their base types without breaking behavior**.
Example: If `Rectangle` is a `Shape`, replacing it with `Square` shouldn't break calculations.

4. I – Interface Segregation Principle (ISP)

- No client should be forced to **depend on methods it doesn't use**.
Example: Split a large interface into smaller, specific ones.

5. D – Dependency Inversion Principle (DIP)

- High-level modules should **not depend on low-level modules**, but both should depend on **abstractions**.
Example: `PaymentService` depends on `IPaymentGateway`, not `PayPalGateway`.

2. What is DRY principle?

- **DRY = Don't Repeat Yourself**
- Avoid duplicating code; reuse logic via functions, classes, or modules.
Example: Use a `CalculateTax()` method instead of writing the same formula in multiple places.

3. What is KISS principle?

- **KISS = Keep It Simple, Stupid**
- Write **simple, readable, maintainable code**. Avoid unnecessary complexity.

4. What is YAGNI principle?



- **YAGNI = You Aren't Gonna Need It**
- Don't implement features **until they are actually required**. Prevents over-engineering.

5. What is the purpose of unit testing?

- Verify **individual units of code** work as expected.
- Helps **catch bugs early** and ensures **code quality**.

6. Difference between unit testing and integration testing

Aspect	Unit Testing	Integration Testing
Scope	Single function/class	Multiple modules working together
Speed	Fast	Slower
Dependencies	Often uses mocks	Uses real dependencies
Goal	Validate logic correctness	Validate system interactions

7. Common unit testing frameworks in .NET

- **MSTest** – Built-in framework from Microsoft
- **NUnit** – Popular and flexible testing framework
- **xUnit** – Modern, widely used, supports parallel testing



8. MSTest

- Official Microsoft testing framework for **.NET projects**.
- Example:

```
[TestMethod]
public void Add_TwoNumbers_ReturnsSum()
{
    Assert.AreEqual(5, Calculator.Add(2,3));
}
```

9. xUnit

- Lightweight, extensible framework.
 - Supports **constructor-based setup** and **parallel tests**.
- Example:

```
[Fact]
public void Add_TwoNumbers_ReturnsSum()
{
    Assert.Equal(5, Calculator.Add(2, 3));
}
```

10. NUnit

- Flexible testing framework with **attributes for setup, teardown, and assertions**.
-



11. What is mocking in unit testing?

- Simulating **dependencies or external services** to test **units in isolation**.
Example: Mock a database call instead of hitting a real DB.
-

12. Moq framework

- Popular .NET **mocking library** to create **fake objects** for testing.
-

13. Test-Driven Development (TDD)

- Write **tests before code**, then implement functionality to **pass the tests**.
 - Workflow: **Red** → **Green** → **Refactor**
-

14. Behavior-Driven Development (BDD)

- Focuses on **behavior specifications** instead of implementation.
 - Uses **Gherkin syntax**: Given-When-Then.
-

15. Continuous Integration (CI)

- Automatically **build and test code on commit**.
Tools: GitHub Actions, Azure DevOps, Jenkins
-



16. Continuous Delivery / Deployment (CD)

- CD = Automates **deployment to staging**
- Continuous Deployment = Automates **deployment to production**

17. What is Git and why is it used?

- Git is a **distributed version control system**.
- Tracks changes, allows collaboration, supports branching and merging.

18. Git vs GitHub

Aspect	Git	GitHub
Type	VCS tool	Hosting service for Git repositories
Function	Track code history	Collaboration, pull requests, CI/CD

19. Common Git commands

Command	Purpose
clone	Copy repo locally
commit	Save changes locally
push	Upload changes to remote
pull	Fetch and merge remote changes
merge	Combine branches



rebase Reapply commits on another branch

20. Branching and merging

- **Branching** – Create a separate line of development.
 - **Merging** – Combine changes from branches.
-

21. Pull request

- Proposal to **merge code from one branch to another**, reviewed by teammates.
-

22. Best practices for writing clean code in C#

- Follow **naming conventions**
 - Keep **methods small and focused**
 - Use **SOLID principles**
 - Avoid **magic numbers**, use constants
 - Comment **why**, not **what**
-

23. Logging and importance

- Record application **activity, errors, and debugging info**



- Tools: Serilog, NLog, Microsoft.Extensions.Logging

24. Exception handling best practices

- Catch **specific exceptions**
- Avoid swallowing exceptions silently
- Use **try-catch-finally** appropriately
- Log errors for troubleshooting

25. Caching and importance

- Store frequently accessed data **temporarily** for faster access.
Example: Cache user profile info in memory instead of DB every time.

26. Dependency Injection (DI)

- Provides **dependencies from outside** instead of creating them inside classes.
Benefits: Looser coupling, easier testing, maintainability.
Example:

```
services.AddScoped<IEmailService, EmailService>();
```

27. Code review and importance

- Review **peer code** to ensure **quality, maintainability, and best practices**



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

- Catches **bugs**, **security issues**, and **inconsistencies**
-

28. Design patterns

- Reusable solutions for **common software problems**
 - Examples: Singleton, Factory, Observer, Repository
-

29. Performance profiling in .NET

- Analyze CPU, memory, and I/O usage
 - Tools: **Visual Studio Profiler**, **JetBrains dotTrace**, **PerfView**
-

30. Importance of documentation

- Ensures **maintainability**
 - Helps **new developers onboard** quickly
 - Reduces **miscommunication in teams**
-



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET,CLOUD,API & REACT)

@Sandeep_pall

<https://www.linkedin.com/in/sandeepall/>

Follow me on Linkedin: <https://www.linkedin.com/in/sandeepall/>